SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT235

Examination - January Semester 2025

Data Structures and Algorithms

Saturday, 19 April 2025 **10:00 am - 12:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **7** questions and comprises **6** printed pages (including cover page).
2. You must answer **7** questions.
3. All answers must be written in the answer book.
4. This is **Closed-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT235 Copyright © 2025 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Analyse the sorting algorithm provided below and answer the following questions.

```
def sort(arr):
    if len(arr) <= 1:
        return arr

    p = arr[len(arr) // 2]

    a = []
    b = []
    c = []
    for x in arr:
        if x < p:
            a.append(x)
        elif x > p:
            c.append(x)
        else:
            b.append(x)

    return sort(a) + b + sort(c)
```

Question 1a

The variable p is defined as p = arr[len(arr) // 2]. What does p represent in the context of this algorithm? Furthermore, explain how p is initialized and utilized during the partitioning process.

(7 marks)

Question 1b

What is the time complexity of this algorithm?

(2 marks)

Question 1c

Explain why this performance is expected.

(7 marks)

Question 2

Analyse the Python implementation of the queue ADT using a singly linked list provided below. Please answer the following questions carefully.

```python
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class LinkedQueue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def is_empty(self):
        return self.size == 0

    def enqueue(self, data):
        new_node = Node(data)
        if self.is_empty():
            pass # Q2(b): Complete the code here
        else:
            pass # Q2(b): Complete the code here
        self.rear = new_node
        self.size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        data = self.front.data
        self.front = self.front.next
        self.size -= 1
        if self.is_empty():
            self.rear = None
        return data
```

Question 2a

Discuss the advantages of implementing a queue using a singly linked list instead of an array, focusing on memory management and time complexity.

(8 marks)

Question 2b

Complete the enqueue function by filling in the missing lines of code, indicated by 'pass # Q2(b): Complete the code here'.

(4 marks)

Question 3

Demonstrate a clear understanding of the tree shown below in Figure Q3 by answering the following questions.

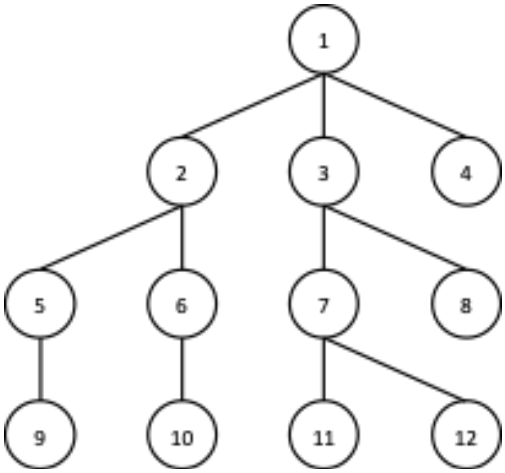

Figure Q3

Question 3a

How many internal nodes are in the tree, and what are they?

(2 marks)

Question 3b

What is the degree of node 7?

(2 marks)

Question 3c

What is the depth of node 9?

(2 marks)

Question 3d

What are the siblings of node 5?

(2 marks)

Question 3e

What is the order of visits in preorder traversal? Analyse its time complexity.

(4 marks)

Question 4

Consider the AVL tree shown below. Insert a node with the key 79, and rebalance the tree if needed. You must illustrate all intermediate steps using diagrams and highlight the balance factor of each node.

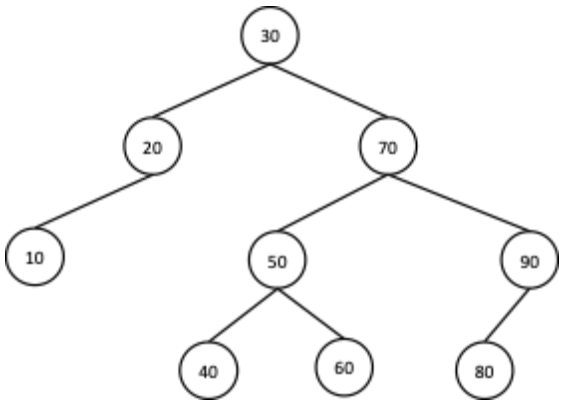

Figure Q4

(10 marks)

Question 5

Consider a scenario where you need to remove the node with the minimum key from a priority queue ADT.

Question 5a

What are the **THREE (3)** steps involved in removing the node with the minimum key?

(3 marks)

Question 5b

What is the name of the algorithm used to restore the heap-order property in a priority queue ADT after the node with the minimum key is removed?

(2 marks)

Question 5c

What is the time complexity of this algorithm? Justify why this performance is expected.

(4 marks)

Question 5d

Illustrate the algorithm identified in Q5(b) using diagrams and provide a textual description of its steps.

(11 marks)

Question 6

Consider a hash table with eight slots (indices 0 to 7) that results from using the hash function h(i) = (2i + 5) mod 8 to hash keys 12, 88, 94, 11, 20, 16, 5, and 7. Assume collisions are handled using linear probing.

Question 6a

Populate the hash table using the template below, illustrating the final state of the table.

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Key								

*** This is the template table.**

(10 marks)

Question 6b

How many collisions were found while solving for Q6(a)?

(2 marks)

Question 7

Please answer the following questions related to the quick sort algorithm.

Question 7a

What are the **THREE (3)** types of pivots used for the quick sort algorithm?

(3 marks)

Question 7b

Label each pivot strategy as either a good or bad pivot.

(3 marks)

Question 7c

Justify your answers to Q7(b). In your explanation, include the time complexity of the quick sort algorithm for each pivot strategy and explain the reasoning behind its expected performance.

(12 marks)

----- END OF PAPER -----